Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATE THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Evaluating the Impact of Agent Harness Components on the Reliability and Efficiency of LLM Agents**

Работу выполнил / Thesis is executed by	**Danila Kochegarov**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Andrey Sadovykh**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2027

Contents

1 Introduction **3**

1.1 Background and Motivation 3

1.2 Models and Deployment Setting 4

1.3 Problem Statement and Research Gap 5

1.4 Aim, Research Questions, and Hypotheses 6

1.5 Planned Approach 8

1.6 Expected Contribution and Scope 9

2 Literature Review **10**

2.1 Scope and Analytical Framework 10

2.2 Agent Interfaces and Tool Design 12

2.3 External Feedback and Error Correction 14

2.4 Model Capability and Harness Benefits 17

2.5 Context Management 19

2.6 Evaluation of Reliability and Efficiency 20

2.7 BPMN Generation in Modelio 25

2.8 Synthesis and Implications for the Study 26

References **29**

Chapter 1

Introduction

1.1 Background and Motivation

An agent harness is the software infrastructure that organizes how a language model interacts with tools and an execution environment. It supplies instructions and observations, manages context and persistent information, and controls when execution continues or stops. In this thesis, an *agent system* comprises the model, its harness, and their interaction with the environment; the harness is the configurable software part of that system. A *harness component* is a part whose behavior can be enabled, disabled, or replaced while the underlying model remains fixed. The research focuses on the contribution of such components, rather than treating the performance of an entire agent system as a property of its model alone.

Large language models (LLMs) generate responses from a supplied context. In an interactive agent system, those responses can request tool actions, whose results become observations for subsequent decisions. ReAct illustrates this interaction between reasoning, actions, and observations [1]. SWE-agent further

shows that the interface through which a model edits files and receives feedback affects task performance [2]. These findings motivate evaluating the surrounding software as an experimental factor.

The initial study selects two components. The first is an *automatic execution-feedback hook*: a programmatic trigger that checks each completed new version of an artifact and returns detected errors to the model. The second is *persistent error–correction memory*: records that associate an observed error with a correction diff, meaning the changes between artifact versions, and the outcome of checking that revision. The hook determines when feedback arrives within a run; memory makes selected experience available in later runs. Their separate and combined effects are the subject of the comparison.

1.2 Models and Deployment Setting

Model preparation and harness design are distinct. A *base model* is trained to predict text continuations and has not necessarily been adapted to follow user requests. *Instruction tuning* further trains a model on examples of desired responses to instructions; subsequent training may also use preference feedback. *Domain specialization*, such as training focused on code, concerns a different dimension. A specialized model may also be instruction-tuned. Ouyang et al. demonstrate that training with human feedback changes instruction-following behavior [3].

The study uses post-trained models intended for instruction-following interaction because its protocol requires following tool descriptions, producing structured actions, and using returned observations. This is a reason to match the model to the protocol, not a claim that instruction-tuned models universally outperform all alternatives with deterministic tools. In particular, instruction following does

not guarantee correct tool use or successful error repair. The selected checkpoint and inference configuration will remain fixed across component comparisons.

The first pilot will use only Qwen3.5-9B in the 8-bit Q8_0 format. Its official model card identifies it as post-trained and documents tool use; a quantized GGUF distribution supplies the selected format [4], [5]. Quantization reduces the numerical precision used to represent weights. The intended configuration is text-only inference fully on a GPU with 16 GB of video memory. This is a deployment target to verify, not an already measured result: context storage and execution buffers consume memory in addition to the weights. Subsequent comparisons may add other model sizes and a cloud-hosted reference. A pilot with one model cannot establish how component effects vary across models.

1.3 Problem Statement and Research Gap

Adding a component does not ensure that a model will use it effectively. Lin et al. distinguish the ability to produce harness updates from the ability to benefit from them. Their findings show a non-monotonic relationship between initial task performance and harness benefit, including failures to activate relevant artifacts or follow them after activation [6]. Automatic checking may address omitted verification, while stored corrections may reduce repeated mistakes; both mechanisms still depend on the model's ability to interpret and apply the available information.

Prior work already studies agent interfaces, externally grounded correction, and experience retained between trials [2], [7], [8]. The unresolved question for this thesis is how automatic feedback and error–correction memory affect technical success and resource consumption, separately and together, under a con-

strained local deployment. Evidence from other tasks motivates this comparison but does not establish its outcome for the selected model and environment.

The first evaluation domain will be generation of Business Process Model and Notation (BPMN) models from natural-language descriptions, using the public Modelio artifact and the PMo scenarios employed by Sadovykh et al. [9]–[11]. A run will count as technically successful when its resulting model opens in Modelio without errors under the defined checking procedure. A script-based adapter will collect the errors exposed by Modelio execution and checking. Structural metrics will be collected separately. This criterion does not establish that every requirement of a process description has been represented correctly; scenario-specific semantic rules and expert judgments are outside the initial evaluation. BPMN supplies the first test domain, and developing a new validator is not the main research objective.

1.4 Aim, Research Questions, and Hypotheses

The aim is to measure how selected harness components affect the reliability and efficiency of LLM agent systems and, in a subsequent comparison, how their effects depend on model capability. The research questions are:

RQ1. How does automatic execution feedback change technical success and resource consumption when the same checking tool is already available for voluntary use?

RQ2. How does access to error–correction memory change performance over successive fresh runs of the same task?

RQ3. How do automatic feedback and memory interact: does their combination provide additional benefit, redundancy, or additional cost?

RQ4. How do component effects vary with a model's initial task performance and deployment requirements when the evaluation is extended beyond the first model?

The general working hypothesis is that component effects depend on the model's initial capabilities. A more specific hypothesis for the later model comparison is that smaller models obtain a larger improvement in technical success. This expectation is not treated as established: the non-monotonic effects reported by Lin et al. motivate the alternative that models with intermediate capabilities benefit most [6]. Deployment size and measured capability will be distinguished. Initial success rates must also be considered, because a model already close to perfect technical success has little room for absolute improvement.

The first pilot addresses narrower hypotheses within Qwen3.5-9B. Automatic feedback is expected to improve technical success relative to otherwise matched conditions without the hook. Access to error–correction memory is expected to improve later-run performance relative to conditions without memory at the same position in the sequence. Whether their combination provides an additional benefit remains an open question. Token use and time will accompany success measurements; a gain in success does not automatically imply improved efficiency. Unchanged or worse results remain informative.

1.5 Planned Approach

One configurable harness will support four conditions: both components disabled, hook only, memory only, and both enabled. The model, tool implementations, task descriptions, artifact format, and stopping rules will be shared. The checking tool will remain available voluntarily in all four conditions, so memory can also receive experience from model-initiated checks when the automatic hook is disabled. Research logs will be collected in every condition; enabling memory changes the agent's access to earlier experience.

Within a memory sequence, each step will be a fresh complete attempt at the same task, starting from restored task files and a clean dialogue. Only the designated memory will carry information forward. Separate sequences will begin with empty memory and will not exchange records. This separates retained experience from simply continuing to edit an earlier solution. A direct-generation baseline remains an additional candidate comparison: the model would produce an artifact without an interactive tool loop, and the same external evaluator would check it afterward. Such a baseline measures the overall interactive setup and does not replace the four component conditions.

All conditions will use the same evaluation procedure and equivalent initial environments. Isolation and recorded software versions support reproducibility but do not, by themselves, guarantee deterministic generation. The pilot will establish the Modelio adapter, verify local inference, and calibrate the experimental budget. Reliability and efficiency analysis will distinguish independent sequences from dependent steps that inherit memory. Detailed budgets, memory-selection rules, and statistical procedures belong to the methodology.

1.6 Expected Contribution and Scope

The expected contribution is a reproducible comparison of two harness components, estimates of their individual and combined benefits and costs, and a trace-based analysis of failures and repeated errors. These are planned outputs, not experimental results. The first pilot will establish feasibility and component behavior on one model; testing the model-size hypothesis requires a later extension.

The initial findings will concern technical acceptance in Modelio. Generalization to other domains will require additional tasks with their own reproducible evaluators. HumanEval+ remains a possible programming benchmark [12], but it would add another technical domain and would not alone establish broad domain coverage. Automatic selection of specialized agents or prompts, model fine-tuning, and components requiring an auxiliary LLM are outside the initial scope. Memory transfer between different tasks is also outside the first within-task comparison.

Chapter 2 reviews the evidence motivating this design. The subsequent methodology chapter will define the experimental protocol. Later chapters will report implementation, results, and limitations once the experiments have been conducted.

Chapter 2

Literature Review

2.1 Scope and Analytical Framework

This chapter reviews research relevant to measuring the effect of agent-harness components. It connects work on agent interfaces, external feedback, persistent experience, reusable skills, context management, and evaluation. The purpose is to establish which mechanisms are supported by prior evidence and which comparisons are needed for the planned experiment. The review is organized by mechanism and evaluation design rather than as a chronological list of systems.

Four distinctions guide the analysis. First, a model's underlying capabilities and the software that exposes those capabilities are separate experimental factors. Second, a tool can be available without being called, and can be called without producing a successful correction. Third, an evaluation procedure can be reproducible while measuring only a limited property of the output. Fourth, an independent repetition and a fresh attempt that inherits memory are different experimental units. These distinctions are especially relevant when the target mod-

els have limited resources and the available evaluator checks technical acceptance rather than complete task semantics.

Instruction-following behavior is a property of model training and should not be conflated with harness design. Ouyang et al. train models using demonstrations and human preference feedback and report that human evaluators prefer their responses to those of the underlying base model on the studied prompt distribution [3]. This supports adapting models to instruction-following interaction. It is not a controlled demonstration that instruction tuning is optimal for every restricted tool set. The present study selects a post-trained checkpoint suited to its interaction protocol and keeps its weights fixed. A comparison with a base checkpoint or a differently specialized model would change an additional factor and would not isolate the harness.

The initial component scope is restricted to mechanisms whose internal logic does not invoke an auxiliary language model. This is an operational boundary, not a claim that every non-LLM program is deterministic: a tool can still depend on randomness, external state, or software versions. Reproducible checking requires control over those dependencies. Similarly, whether an instruction was originally written by a human or generated by a model is different from whether its runtime behavior is fixed. The experiment will document the component's behavior and dependencies rather than infer determinism from its provenance.

2.2 Agent Interfaces and Tool Design

2.2.1 Interaction Between Reasoning and Action

ReAct, introduced by Yao et al., interleaves reasoning with actions and observations [1]. An action can obtain information from an external environment, and the resulting observation can influence the next decision. This provides a conceptual basis for comparing direct generation with an interactive agent: the latter can change its output after encountering evidence unavailable at the start of generation.

However, the addition of interaction changes several factors simultaneously. It introduces tool access, additional observations, and potentially additional inference steps. A higher success rate under an interactive configuration therefore does not, by itself, identify the contribution of one component. A direct-generation condition can provide a practical reference, but the main component comparison requires matched interactive conditions with the hook and memory switched independently. The distinction is necessary to avoid attributing all benefits of iteration to the hook alone.

2.2.2 Interfaces That Reduce Avoidable Errors

SWE-agent studies the agent-computer interface as a design problem [2]. Its interface supports repository navigation, file editing, and execution, and supplies concise feedback about actions. In particular, its editing workflow incorporates linting so that selected errors can be reported when an edit is attempted. The paper evaluates alternative interface choices and shows that tool presentation and feedback influence agent behavior.

This is directly relevant to the proposed execution-feedback component. Error detection can be placed in the surrounding software rather than left entirely to a model's decision to check its work. Nevertheless, SWE-agent's results concern its particular software-engineering environment and evaluated models. They establish a precedent for automatic checks, not an expected effect size for BPMN generation. They also mean that the novelty of this thesis must lie in its controlled empirical comparison and deployment setting, rather than in the general idea of returning errors automatically.

AgentOccam approaches the same problem through observation and action-space design [13]. Its web agent simplifies the representations exposed to the model and removes unnecessary interaction choices. The reported WebArena improvements indicate that a more elaborate agent architecture is not always required for better performance. Aligning the interface with what a model can use effectively can be more important than increasing the number of available tools.

For the present study, this supports starting with a minimal, well-documented tool set. It does not establish that reducing information or tool access is always beneficial. Web observations, repository files, and BPMN construction errors have different structures, and the information needed to correct one task may be redundant for another. Interface simplification should therefore be treated as an experimentally testable choice.

2.2.3 Fixed Workflows as a Comparison Point

Agentless provides another challenge to the assumption that increased agent autonomy necessarily improves results [14]. It organizes software repair into localization, repair, and patch validation, rather than giving the model unrestricted

control over an evolving sequence of tools. Its results demonstrate the value of a simple and inspectable workflow on the evaluated software-repair benchmark.

The relevant distinction is the allocation of control. In a model-directed agent, the model chooses whether to inspect, execute, or revise. In a fixed workflow, some of those decisions are made by the surrounding program. The planned automatic hook will move one decision—when to run an execution check—into the harness, while leaving the generation of a correction to the model. Agentless motivates examining that allocation of control, but does not answer whether the particular hook will improve the selected models.

Together, these studies suggest that tool availability, interface representation, and execution policy should be separated in the experiment. If a condition also receives a better helper library or more detailed tool documentation, an improvement would be ambiguous. The four component conditions will therefore share those resources. Access to accumulated error–correction records is the intended memory intervention, while the automatic-trigger policy is the intended hook intervention.

2.3 External Feedback and Error Correction

2.3.1 Limits of Intrinsic Self-Correction

Huang et al. investigate intrinsic self-correction, in which a model revises a response without external feedback [15]. On their reasoning tasks, self-correction often fails to improve the response and can reduce performance. Their findings challenge the assumption that simply asking a model to reconsider its answer is a reliable error-correction mechanism.

The scope of that conclusion matters. It concerns the evaluated models, tasks, and correction procedures; it is not a proof that every form of self-correction is impossible. For the proposed experiment, the useful implication is that an additional model call should not automatically be counted as a meaningful verification step. Feedback from executing a generated artifact supplies evidence about its behavior in the target environment, whereas an unsupported request to reflect may only produce another plausible explanation.

2.3.2 Tool-Supported Critique

CRITIC, proposed by Gou et al., combines language-model revision with feedback obtained through external tools [7]. Starting from an initial output, the system interacts with a relevant tool, evaluates aspects of the output, and then revises it. The paper reports improvements across question answering, mathematical program synthesis, and toxicity reduction. Its central contribution to this review is the separation between generating a critique and grounding that critique in external evidence.

CRITIC does not imply that every external signal is complete, deterministic, or sufficient for a correct answer. A tool only reveals the properties it can inspect. In Modelio, an execution error can expose an invalid API call or an unsuccessful construction step, but absence of such an error does not establish that the generated process expresses the intended business logic. The proposed hook will use execution evidence within this limited role.

The study will also distinguish feedback availability from feedback interpretation. Automatically returning an error removes the need for the model to request the check, but the model must still understand the message and construct a useful

revision. Thus, automatic checking may reduce failures caused by omitted verification while leaving failures caused by poor correction ability unchanged. This is a plausible mechanism to examine in traces, rather than an outcome to assume in advance.

2.3.3 Reflection and Persistent Experience

Reflexion, introduced by Shinn et al., stores language-model-generated reflections on feedback in an episodic memory and uses them in subsequent trials [8]. It demonstrates adaptation through retained information without updating model weights. The feedback may come from an external evaluator or internal simulation. This provides a precedent for studying experience across attempts, while distinguishing the stored information from the model parameters.

The proposed memory component uses a more constrained representation: an observed error, the diff associated with a revision, and the outcome of checking that revision. It does not require an auxiliary model to generate advice. A diff is evidence of a change, not proof of a successful repair; the checking outcome is needed to distinguish confirmed corrections from unsuccessful revisions. This representation is a design choice for the present study, not a result already established by Reflexion.

Automatic feedback and memory address different parts of the interaction. The hook exposes a problem during the current run. Memory may make a previously observed problem and correction available before the same mistake recurs. Their combination could be complementary, but could also add redundant context or preserve misleading examples. This motivates evaluating all four combinations rather than attributing any improvement in the combined system to memory alone.

The comparison will retain experience only within a sequence of fresh attempts at the same task. Task files and the dialogue will be reset before each attempt; separate sequences will start with empty memory. Improvement under this protocol concerns repeated-task adaptation. It does not establish transfer to unseen tasks, and its dependent attempts cannot be counted as independent reliability trials.

2.4 Model Capability and Harness Benefits

Lin et al. distinguish *harness-updating capability*, the ability to produce useful persistent changes, from *harness-benefit capability*, the ability to use those changes during task solving [6]. In their setting, models with intermediate initial performance benefit most; weaker models benefit less because of failures to activate or follow artifacts, while stronger models show smaller gains consistent with a performance ceiling. Their initial capability measure concerns performance before harness evolution, not necessarily generation without an interactive harness. These are capability groups in the studied tasks, not GPU-memory classes.

This evidence motivates an alternative to the expectation that smaller models gain more. A weak baseline leaves room for improvement but does not establish an ability to use feedback or memory. Conversely, high initial success limits the attainable absolute increase. The later model comparison should therefore report initial success alongside the change under each component configuration. Parameter count, deployment memory, and task performance should remain separate descriptors. The first single-model pilot can estimate component effects but cannot resolve their relationship with model size.

SkillsBench provides complementary evidence through paired evaluation of reusable skills [16]. The reviewed version evaluates 87 tasks in eight domains under matched no-skill and curated-skill conditions across 18 model-harness configurations. It reports an increase in average pass rate from 33.9% to 50.5%, or 16.6 percentage points. Variation between configurations shows why an aggregate benefit cannot determine the outcome for a particular model.

A skill supplies procedural instructions and supporting resources, whereas a hook changes execution policy and error–correction memory supplies experience from prior attempts. SkillsBench motivates matched comparisons, but does not identify the effect of either proposed component. In the present study, the underlying tool documentation and memory-management implementation will remain fixed while the hook and access to accumulated records are varied. The contents of enabled memory will evolve through the sequence by design.

2.4.1 Deployment Resources and Model Selection

Resource categories describe how a model is deployed, rather than a universal ranking of its abilities. Phone-scale shared-memory execution, a consumer GPU, a server GPU, and a multi-GPU installation differ in both capacity and execution characteristics. Comparisons need the actual memory budget and inference settings; a category name alone does not make configurations comparable. The first pilot uses a concrete 16 GB GPU target instead of assuming a standard numerical boundary for every model class.

Memory use includes weights, context-related state, and runtime buffers. Quantization changes the representation of weights, while context length and simultaneous requests affect additional memory use. Moving part of the computa-

tion or model state to CPU memory would constitute a different deployment configuration. Consequently, a small weight file does not by itself prove that an agent workload fits fully on the GPU. Peak memory during realistic tool interactions is the relevant feasibility check.

Qwen3.5-9B is selected as the first post-trained checkpoint, with Q8_0 as the chosen weight format [4], [5]. Its documented tool-use interface motivates a pilot; it is not evidence of successful integration with the proposed harness. The final inference engine, context settings, and reasoning mode must be held constant across component conditions. Expanding to multiple checkpoints will require accounting for differences in training and quantization as well as nominal size.

2.5 Context Management

An agent's history grows as it receives tool outputs and produces further actions. Context-management methods address both the cost of that history and the risk that important information becomes difficult to use. LongLLMLingua uses model-based importance estimates to compress prompts while preserving information relevant to a query [17]. Its results show that the quantity and arrangement of retained information can affect both task performance and resource use.

LongLLMLingua is relevant as a comparison mechanism, but its compression process uses an auxiliary language model. It is therefore outside the initial scope of components that require no auxiliary language model. This exclusion is a design boundary for the first experiment, not a judgment that model-based compression is ineffective. Any later comparison should include the compression model's own cost rather than count only the tokens saved at the main model.

Lindenbauer et al. compare observation masking with LLM-generated summaries for agent context management [18]. Their study evaluates configurations of SWE-agent and includes an additional probe with OpenHands. It finds that replacing older observations with a simpler representation can substantially reduce cost while achieving solve rates comparable to summarization in the evaluated settings. The work appeared in the DL4C workshop associated with NeurIPS 2025; it should be distinguished from a main-conference paper.

The findings motivate simple context policies as candidates for later experiments. They do not justify assuming that shorter context always improves a model, or that results from one harness transfer unchanged to another. Both feedback hooks and persistent memory can expand context by adding errors and previous revisions. The initial comparison will therefore keep the policy for handling the current dialogue constant and record the cost of supplied memory separately where possible. The memory representation and selection policy must also remain consistent between memory-enabled conditions. This prevents a simultaneous change in summarization from being mistaken for a benefit of feedback or retained experience.

2.6 Evaluation of Reliability and Efficiency

2.6.1 Task Outcomes and Evaluator Coverage

AgentBench evaluates LLMs across eight interactive environments and identifies difficulties in long-term reasoning, decision-making, and instruction following [19]. It establishes that agent performance is multidimensional and depends on the task environment. Its broad model comparisons are useful context,

but they do not replace a controlled comparison of individual harness components within one model.

AppWorld provides a different contribution: an executable environment in which agents interact with applications through APIs and are evaluated using state-based unit tests [20]. The evaluation can accept different action sequences that reach the required state and can check for unintended changes. This shows how programmatic evaluation can be tied to task outcomes rather than to the surface form of a generated answer.

The planned Modelio criterion is narrower than AppWorld's task-specific tests. It will establish that the generated artifact can be opened without errors under a defined procedure. It will not establish all required business-process behavior. A reproducible acceptance rule is still useful for the present question, provided its coverage is explicit. The reported metric will therefore be described as *technical success*, and comparisons with benchmarks using stronger functional tests will remain qualitative.

Structural counts require a similar distinction. Numbers of elements, gateways, or sequence flows describe a generated model, but a match in these counts does not imply that the connections or process meaning are correct. They will be recorded as descriptive measurements and possible indicators of unusual outputs. They will not be combined into an unvalidated score or used as additional pass/fail conditions in the first experiment. In particular, a diagram that opens successfully may still omit required activities or branches. The collected counts may reveal such cases for inspection, but the initial success criterion will not silently be strengthened after observing them.

2.6.2 Task Diversity and Candidate Extensions

HumanEval contains 164 function-level programming tasks with executable tests [21]. EvalPlus expands the tests and shows that additional inputs expose incorrect programs accepted by the original suite [12]. HumanEval+ could therefore complement Modelio's technical acceptance criterion with functional evaluation. However, it remains a programming benchmark and would add a second technical domain rather than establish broad coverage.

AppWorld and τ-bench illustrate other application settings, including personal application workflows and service interactions evaluated against target states [20], [22]. They offer directions for broader task selection, but their simulators, interfaces, and evaluation costs would need to be assessed before inclusion. No second dataset is required for the first pilot. A later domain should provide reproducible automatic outcomes and allow the same component questions to be studied without introducing scenario-specific expert judgments.

Each domain requires its own evaluator behind a common experimental interface. A single BPMN validator cannot evaluate unrelated application tasks. Within a domain, all conditions must produce compatible artifacts or final states and use the same acceptance rule. Results should be reported per domain rather than pooled as though all success criteria had equivalent coverage.

An interactive adaptation also needs to distinguish feedback visible during task solving from final evaluation. Neither HumanEval nor its expanded tests defines an agent's correction loop. Public-task familiarity, access to tests, and possible performance ceilings affect interpretation; an interactive score should not be equated with standard one-shot benchmark performance. These considerations

concern the selection and adaptation of a later benchmark, rather than a claim that it has already been selected.

2.6.3 Repeated Runs and Consistent Success

Yao et al. introduce τ-bench to evaluate agents interacting with users and tools under domain rules [22]. Its outcome evaluation compares the final database state with an annotated goal state. It distinguishes obtaining at least one success across several trials, $\mathrm{pass@}k$, from succeeding consistently in all of them, pass^k. This distinction matters when an agent is expected to handle repeated requests reliably rather than generate several candidates from which a successful one can be selected.

The interpretation depends on how trials are initialized. Interaction rounds within one attempt are not independent complete trials. Likewise, attempts that inherit error–correction memory do not represent repeated draws from an unchanged initial state. Applying an independent-run consistency measure to the successive steps of a memory sequence would confuse accumulated experience with repeated-run reliability.

For the proposed memory comparison, results at the same step should be compared across separately initialized sequences and matched component conditions. A trajectory can show whether success, cost, or recurrence of errors changes as experience accumulates. Uncertainty analysis must account for dependence within a sequence and shared task difficulty. The exact repetition counts and statistical estimators belong in the methodology; they cannot be inferred from the number of interaction rounds or memory records.

2.6.4 Cost and Attribution

For two conditions A and B, the change in technical success will be reported as $\Delta\mathrm{SR} = \mathrm{SR}_B - \mathrm{SR}_A$, expressed in percentage points when rates are presented as percentages. This will be accompanied by token usage, elapsed time, model calls, and execution attempts. Resource totals will include failed runs; excluding them would hide an important cost of an unreliable configuration.

Cost per successful task can be informative, but it becomes undefined when a condition has no successes. In that case, total resource use and the zero-success count should be reported directly. Likewise, additional tool calls are not inherently beneficial. A hook may execute more checks without improving final success, while a growing memory may add input tokens without preventing repeated errors. Either component may improve success only at a substantial inference cost. The interpretation should preserve that trade-off.

The direct-generation baseline and interactive conditions offer different opportunities for correction. Their comparison measures a practical change in the execution setup, not a pure interface effect at identical realized computation. Within the four-condition design, the hook effect can be compared with memory disabled and with memory enabled; the memory effect can likewise be compared at each hook setting. A difference between those conditional effects indicates an interaction. Common tools, representations, and stopping rules support attribution, while the extra computation and context induced by each component remain part of its cost.

A stopping rule also changes the interpretation of an outcome. Reaching a round limit or a time deadline is not, by itself, evidence of an infinite loop. Repeated actions, recurring diagnostics, and unchanged artifacts can provide

descriptive evidence of cycling. A timeout records failure to complete within the permitted interval; its duration is not a measured time to eventual success. Cloud and local elapsed times additionally reflect serving infrastructure, so cross-deployment speed differences cannot be attributed solely to model size.

Traces can connect aggregate measurements to possible mechanisms: whether checking was requested, whether an initial error was corrected, and how many attempts preceded completion. These observations support diagnosis, but do not independently establish causality. For example, a positive association between token use and failure may reflect difficult tasks requiring more attempts. The experimental protocol must define the observations and denominators before interpreting such relationships.

2.7 BPMN Generation in Modelio

Sadovykh et al. study LLM-assisted BPMN construction in Modelio through direct API generation and a helper-supported intermediate representation [9]. Their comparison uses three LLMs and 55 scenarios from the PMo dataset, originally released by Brissard, Cuppens, and Zouaq [10]. The artifact supplies a first evaluation domain and illustrates the relevance of tool interfaces. The present component comparison will hold the helper interface and output representation fixed, so their effects are not mixed with those of feedback or memory.

The public helper script `BPMN_Helpers.py` constructs models and diagrams. A separate script, `get_bpmn_metrics.py`, collects structural counts [11]. Neither provides complete semantic validation. A script-based execution adapter will capture the errors exposed by Modelio and return them in a consistent format.

Relevant categories must be distinguished: syntax errors in the generated script, exceptions during script execution or Modelio API calls, and model diagnostics exposed by an invoked built-in check. These are not collectively described as diagram compilation errors. The adapter will not invent additional scenario-specific semantic rules.

The reproduction guide identifies an internal ScriptServer, used for automated execution in the original study, that is absent from the public release [23]. Thus, the public generation and metrics scripts do not establish that unattended checking or capture of every Modelio diagnostic is already available. The pilot must verify the execution adapter and the observable error channels before the acceptance procedure is fixed. The planned outcome remains a model that opens in Modelio without errors under that procedure; counts are supplementary measurements, not additional success gates.

2.8 Synthesis and Implications for the Study

Table 2.1 summarizes the relationship between the main strands of literature and the planned comparison. The studies provide evidence for the importance of interfaces, feedback, retained experience, and controlled evaluation. They also constrain the claims that can be made: automatic checks are established practice, published component benefits are setting-dependent, and a reliable evaluator must be interpreted according to what it actually checks.

TABLE 2.1
Prior evidence and its role in the proposed study

Work	Relevant mechanism	Implication and boundary
ReAct [1]	Alternating actions and observations	Motivates an interactive baseline; interaction changes more than one factor.
SWE-agent [2]	Interface design and automatic editing checks	Provides a precedent for hooks; does not determine their effect on local BPMN agents.
AgentOccam and Agentless [13], [14]	Simplified interfaces and constrained workflows	Motivate a minimal tool set and explicit allocation of execution control.
Huang et al. and CRITIC [7], [15]	Intrinsic versus externally grounded correction	Motivate execution evidence while leaving correction ability as an empirical question.
Reflexion [8]	Reflections retained across trials	Motivates memory without weight updates; the proposed records use errors and diffs, not generated reflections.
Lin et al. [6]	Non-monotonic harness benefit	Motivates testing capability dependence and ceiling effects; capability tiers are not memory classes.
SkillsBench [16]	Paired evaluation of skill access	Supports matched component comparisons; skills differ from execution-trigger policies.
AppWorld and τ-bench [20], [22]	Programmatic outcomes and repeated-run reliability	Motivate domain-specific scoring; memory-dependent steps need a separate analysis.
Sadovykh et al. [9]	Helper-based BPMN construction in Modelio	Supplies the first domain and artifacts; automation and semantic evaluation have separate limits.

The reviewed evidence does not determine the individual and combined effects of automatic execution feedback and structured error–correction memory for the selected local model and Modelio tasks. A four-condition comparison addresses that question while keeping the model and tool interface fixed. Separately

initialized sequences allow accumulated experience to be studied without treating their dependent steps as independent runs.

The first pilot will establish feasibility and measure component behavior on Qwen3.5-9B. A later extension to other models is necessary to test whether smaller models benefit more or whether intermediate capability provides the largest gain. Additional task domains would be needed to assess generalization beyond Modelio. The contribution is therefore a controlled empirical comparison with explicit evaluation limits, rather than a new validator or an assumption that more harness functionality is always beneficial.

References

[1] S. Yao, J. Zhao, D. Yu, *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629.

[2] J. Yang, C. E. Jimenez, A. Wettig, *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf.

[3] L. Ouyang, J. Wu, X. Jiang, *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.

[4] Qwen Team. "Qwen3.5-9B: Model card." (2026), [Online]. Available: https://huggingface.co/Qwen/Qwen3.5-9B (visited on 11/01/2026).

[5] Unsloth. "Qwen3.5-9B GGUF: Q8_0 weights." (2026), [Online]. Available: https://huggingface.co/unsloth/Qwen3.5-9B-GGUF/blob/main/Qwen3.5-9B-Q8_0.gguf (visited on 11/01/2026).

[6] M. Lin, J. Wu, Z. Wang, *et al.*, "Harness Updating Is Not Harness Benefit: Disentangling Evolution Capabilities in Self-Evolving LLM Agents," in *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, 2026, pp. 52 829–52 852. DOI: 10.18653/2026.emnlp-main.2326. [Online]. Available: https://aclanthology.org/2026.emnlp-main.2326/.

[7] Z. Gou, Z. Shao, Y. Gong, *et al.*, "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2305.11738.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[9] A. Sadovykh, L. Chlih, B. Said, and H. Bruneliere, "Towards LLM-Assisted Business Process Modeling in an Industrial Modeling Tool: An Experience Report," in *ACM/IEEE 29th International Conference on Model Driven Engineering Languages and Systems*, 2026, pp. 188–199. DOI: 10.1145/3822455.3830328. [Online]. Available: https://hal.science/hal-05702678v1.

[10] A. Brissard, F. Cuppens, and A. Zouaq, *PMo Dataset*, Version 1.0.0, data set, 2025. DOI: 10.5281/zenodo.15857589. [Online]. Available: https://zenodo.org/records/15857589.

[11] Modelio R&D. "Modelio BPMN Automation: MODELS 2026 artifact." Public repository inspected at commit 2debf4e. (2026), [Online]. Available: https://github.com/Modelio-R-D/Modelio-BPMN-Automation-MODELS26 (visited on 10/30/2026).

[12] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://openreview.net/forum?id=1qvx610Cu7.

[13] K. Yang, Y. Liu, S. Chaudhary, *et al.*, "AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=oWdzUpOlkX.

[14] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-Based Software Engineering Agents," *Proceedings of the ACM on Software Engineering*, 2025. DOI: 10.1145/3715754. [Online]. Available: https://dl.acm.org/doi/10.1145/3715754.

[15] J. Huang, X. Chen, S. Mishra, *et al.*, "Large Language Models Cannot Self-Correct Reasoning Yet," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.01798.

[16] X. Li, Y. Liu, W. Chen, *et al.*, *SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks*, arXiv:2602.12670, version 4. Accepted

at NeurIPS 2026, 2026. [Online]. Available: https://arxiv.org/abs/2602.12670v4.

[17] H. Jiang, Q. Wu, X. Luo, *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 1658–1677. DOI: 10.18653/v1/2024.acl-long.91. [Online]. Available: https://aclanthology.org/2024.acl-long.91/.

[18] T. Lindenbauer, I. Slinko, L. Felder, E. Bogomolov, and Y. Zharov, "The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management," in *Fourth Workshop on Deep Learning for Code, co-located with NeurIPS*, 2025. [Online]. Available: https://arxiv.org/abs/2508.21433v3.

[19] X. Liu, H. Yu, H. Zhang, *et al.*, "AgentBench: Evaluating LLMs as Agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2308.03688.

[20] H. Trivedi, T. Khot, M. Hartmann, *et al.*, "AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 16 022–16 076. DOI: 10.18653/v1/2024.acl-long.850. [Online]. Available: https://aclanthology.org/2024.acl-long.850/.

[21] OpenAI, *HumanEval dataset*, Public dataset viewer, 164 tasks. [Online]. Available: https://huggingface.co/datasets/openai/openai_humaneval (visited on 10/30/2026).

[22] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=roNSXZpUDN.

[23] Modelio R&D. "Modelio BPMN Automation: Reproducing the experiment." Public repository inspected at commit 2debf4e. (2026), [Online]. Available: https://github.com/Modelio-R-D/Modelio-BPMN-Automation-MODELS26/blob/main/REPRODUCE.md (visited on 10/30/2026).